

BI2025 Experiment Report - Group 061

Tim Greß*
TU Wien
Austria

Stefan Merdian†
TU Wien
Austria

Abstract

This report documents the machine learning experiment for Group 061, following the CRISP-DM process model.

CCS Concepts

• **Computing methodologies** → **Machine learning**.

Keywords

CRISP-DM, Provenance, Knowledge Graph, Machine Learning

ACM Reference Format:

Tim Greß and Stefan Merdian. 2025. BI2025 Experiment Report - Group 061. In *Proceedings of Business Intelligence (BI 2025)*. ACM, New York, NY, USA, 10 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Code repository: The project code and notebook are available at <https://github.com/timg4/BusinessIntelligence/>.

1 Business Understanding

1.1 Data Source and Scenario

We use the "NBA Players" dataset from Kaggle

(<https://www.kaggle.com/datasets/justinas/nba-players-data>) provided by Justinas Cirtutas. It contains one row per player per season for the NBA seasons from 1996 to 2022. It includes biographical attributes, biometric attributes and basic box score statistics such as games played, average number of points or assists.

In the scenario we assume that the analytics department of an NBA team wants to have decision support for player acquisitions for the next season. The team wants to estimate how impactful a player will be in a season, capture by *net_rating*, which is the difference of points scored and points allowed per 100 possessions when the player is on the court. The historical data will be used to build a model which helps to identify players that are undervalued and helps to avoid signing player with low expected impact.

1.2 Business Objectives

1. Identify players that are undervalued in terms of their expected impact on the team's performance for the next season. 2. Avoid signing players with low expected impact to optimize the teams

*Student A, Matr.Nr.: 12412672

†Student B, Matr.Nr.: 12433732

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

BI 2025, 3

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/XXXXXXX.XXXXXXX>

performance and transfer budget utilization. 3. Assist subjective scouting with a quantitative model and make the decisions more interpretable and reproducible.

1.3 Business Success Criteria

- (1) On the test data, at least 70% of the true top 25% players by *net_rating* should be correctly identified within the model's predicted top 25%.
- (2) Within the top 25% of players predicted by the model, at least 80% should have an above league average *net_rating* in the following season.
- (3) Overall, the model should reduce the impact of bad signings (players with *net_rating* below league average) by at least 25% compared to random or purely subjective signing decisions.

1.4 Data Mining Goals

Build a predictive model to support player acquisition decisions. The main goal is to predict a player's impact using *net_rating* as a regression target. The model outputs a predicted *net_rating* as well as a ranking or shortlist of high-impact candidates (e.g., top 25% by predicted *net_rating*). Season statistics and basic player attributes are used as features, while *player_name* is excluded to avoid identification and leakage issues. Model evaluation follows a time-aware setup, training on earlier seasons and testing on later seasons to reduce look-ahead bias.

1.5 Data Mining Success Criteria

Success is measured on an unseen test set. The model must beat a simple baseline (predicting mean *net_rating*) using RMSE/MAE. Additionally, shortlist quality should be good: selecting the top 25% by predicted *net_rating* should capture most of the true top performers (high recall in top quartile) and the shortlist should have above-average *net_rating* on average.

1.6 AI risk aspects

- (1) Bias via proxy variables such as country or draft status and underrepresented groups.
- (2) Selection bias, as the dataset only includes players who reached the NBA.
- (3) Concept drift over time due to changes in playing style and game dynamics.
- (4) Automation bias if scouts or decision makers over-trust model predictions.
- (5) Privacy and identification risks when using *player_name*.

Mitigation measures include excluding direct identifiers, using time-based train-test splits, and evaluating performance across relevant groups (e.g., USA vs non-USA, drafted vs undrafted).

2 Data Understanding

Dataset Description: The NBA dataset has 12844 rows and 22 columns, 2 of them of type integer, 12 of type float and 8 categorical. The only missing values can be found in the column college, which is to be expected for players without college background (see data source <https://www.kaggle.com/datasets/justinas/nba-players-data>). All other columns are clean.

The following features were identified in the dataset and can be seen in Table 1.

Table 1: Raw Data Features

Feature Name	Data Type	Description
Unnamed: 0	Integer	Original row index from the Kaggle CSV file.
player_name	String	Player full name.
team_abbreviation	String	Abbreviation of the NBA team for that season (e.g. LAL, BOS).
age	Numeric	Player age in years during the given season.
player_height	Numeric	Player height in centimetres.
player_weight	Numeric	Player weight in kilograms.
college	String	College or university the player attended.
country	String	Country associated with the player (typically nationality).
draft_year	String	Year the player was drafted, or <i>Undrafted</i> if not applicable.
draft_round	String	Draft round in which the player was selected, or a code if not drafted.
draft_number	String	Overall draft pick number, or a value for undrafted players.
gp	Integer	Games played by the player in that season.
pts	Numeric	Points per game in that season.
reb	Numeric	Rebounds per game in that season.
ast	Numeric	Assists per game in that season.
net_rating	Numeric	Net rating (offensive rating minus defensive rating) for the player in that season.
oreb_pct	Numeric	Offensive rebound percentage for the player.
dreb_pct	Numeric	Defensive rebound percentage for the player.
usg_pct	Numeric	Usage percentage: estimate of team plays used by the player while on the floor.
ts_pct	Numeric	True shooting percentage for the player in that season.
ast_pct	Numeric	Assist percentage: estimate of teammate field goals assisted by the player.
season	String	Season identifier (e.g. 1996–97, 2019–20).

2.1 Exploratory Data Understanding Steps

2.1.1 Numeric distribution summary. Here we inspect the distributions and plausible value ranges for the numeric NBA features by computing summary statistics for all of them and by computing and IQR-based outlier range and count how many of the values fall out of this range. We also create some histograms for the most important attributes for visualization.

The numeric distribution shows mostly plausible ranges for the biometric attributes but some of the performance metrics seem weird. They show heavy tails and many IRQ outliers, especially `net_rating`, `ast`, `reb`... This issue might arise as we also have players that only played very few games. Histogram (Figure 1) shows distribution of our target variable.

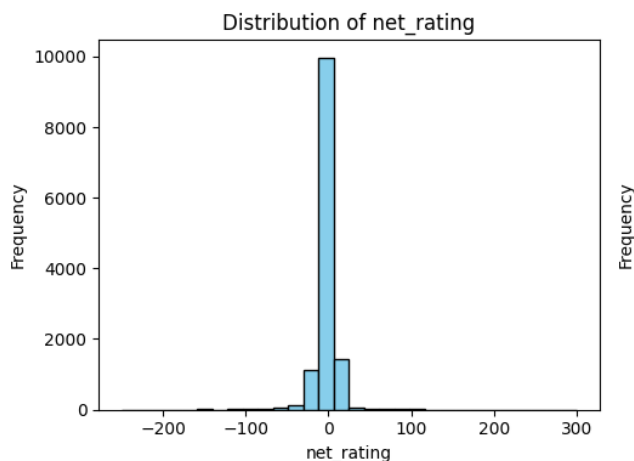


Figure 1: Numerical Distribution of `net_rating`

2.1.2 Categorical distribution report. Compute the categorical frequency distributions for the categorical columns, generates report for the top-5 values per column and visualizes the top 3 for selected columns. We see here that we have 36 different teams, 27 seasons and 2551 unique players. Country is heavily imbalanced towards the US. We have the category "undrafted" in draftrounds and the college has some "nan" values which are just non-college players as we can see in Figure 2.

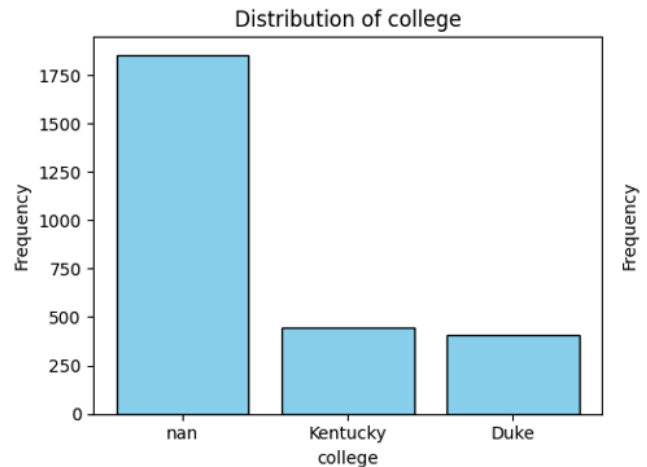


Figure 2: Categorical Distribution of College

2.1.3 Test of Hypothesis. Here we tested the hypothesis whether the outliers in `net_rating` in our data mostly come from players that have a very low number of games.

The hypothesis was tested. Our graphs show that the outliers of `net_rating` are mostly driven by players with a low amount of games because these rate based metrics are very unstable for a small amount of games. We see that the outlier rate for players with 1-5 games is about 0.7% compared to less than 0.05% for players with more than 21 games. This is important information for future steps. The plot 3 shows that extreme `net_rating` values occur almost exclusively for players with very few games played, while `net_rating` stabilizes and outliers largely disappear as games played increase.

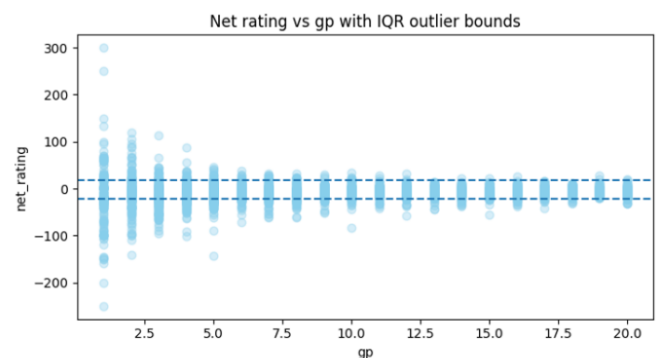


Figure 3: Low games hypothesis with IQR

2.1.4 Correlation Analysis Report. Computed Pearson correlation matrix for numeric attributes and extracted top absolute correlated attribute pairs. Visualized the correlation matrix as a heatmap. There are strong positive correlations between player height and weight, points and assists etc. Surprisingly there is a high negative correlation between a player's height and assists. The heatmap in Figure 4 illustrates the specific correlations very well.

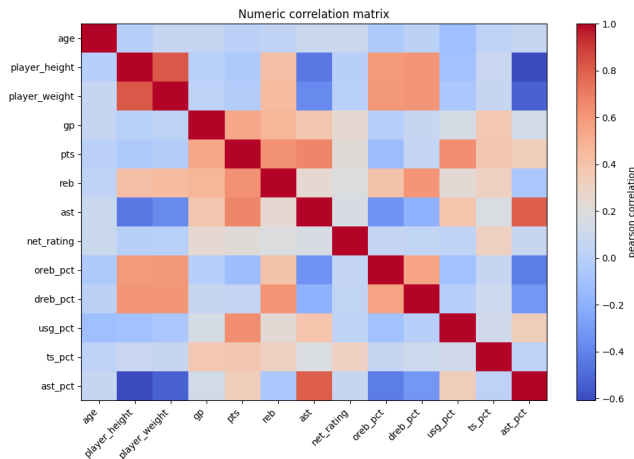


Figure 4: Correlation Matrix for NBA dataset

Ethical Check. There are several things that have to be minded when dealing with this dataset when it comes to ethical concerns. There are no explicit sensitive attributes but some attributes can still act as proxies:

- **country:** this is not necessarily nationality but correlates strongly with a player's origin and their ethnic background and may be used to infer sensitive characteristics
- **college:** this encodes the education pathway and may give hints about socioeconomic differences and scouting pipelines
- **player_height, player_weight:** the distribution is obviously heavily skewed towards large and heavy players, as this is beneficial for basketball. These biometric attributes may enable stereotyping or discrimination as well.
- **player_name:** although these players are public figures, the names still personally identify them and have to be handled responsibly

Underrepresented groups:

- due to the nature of basketball, tiny people and lightweight people are underrepresented in this dataset
- country is heavily imbalanced towards USA players, meaning non-USA players are a minority group in this data representation
- college has a substantial NaN fraction, which likely reflects alternative pathways

For further evaluation we have to mind that we should not only measure performance overall across groups but also within groups (USA vs non-USA etc.) to detect potential bias.

2.2 Potential Biases and Risks

Potential Biases and Risks include the following:

- **Selection Bias:** This dataset only contains players that actually made it to the NBA, which is a highly selective group.
- **Time Bias:** the overall game of basketball evolved over time (for example 3 point shooting increased a lot) so relationships between metrics across seasons are hard to interpret.

- Low sample size for some groups: (international players, undrafted players) can lead to unreliable statistics for these groups
- Players with low games: have heavy skews in rate based metrics (net_rating etc.) which can distort analysis if not handled properly

Questions for Expert:

- Exact calculation of our measurements? Is garbage time included? Is it comparable to other calculation methods?
- What if players are traded mid season? Is that represented as two separate entries or one?
- Are there any known data quality issues with the source data? Is missing college truly just non-college players?

2.3 Preparation actions

Here are some potential data preparation actions based on the data understanding phase:

- handle low games played:
 - filter out players with very low games played as they lead to unstable metrics
 - filter out players with gp under a certain number (e.g. under 20) or weighting by gp
- remove non-informative technical columns
 - drop unnamed_0 column
- handle categorical variables:
 - one-hot encode team_abbreviation, college, country, draft_year, draft_round, draft_number
 - consider embedding for player_name if used
- mind that players can appear across seasons:
 - player_name + season forms a composite key, avoid putting too much weight on one player alone
- scaling of numeric variables:
 - standard scaling or min-max scaling for numeric variables like age, player_height
- handle missing data
 - as the missing rows in college are likely non-college players, we can impute them with a special category no_college

3 Data Preparation

3.1 Data Pre-processing

- (1) Removed the technical index column *Unnamed: 0*.
- (2) Imputed missing values in *college* (NaN) with the category *no_college*.
- (3) Mitigated unstable extreme *net_rating* values caused by very low numbers of games played by filtering to $gp \geq 6$ (parameter $gp_min = 6$).
- (4) Parsed season strings into *season_start* (integer) to enable time-aware train-test splitting.
- (5) Normalized *draft_year*, *draft_round*, and *draft_number* as categorical variables, mapping missing or undrafted values to *undrafted*.

The resulting dataset *:cleaned_data* is derived from the original dataset *:data*.

Table 2: Overview of dataset attributes after data pre-process

#	Column	Non-Null Count	Data Type
0	player_name	12119	object
1	team_abbreviation	12119	object
2	age	12119	float64
3	player_height	12119	float64
4	player_weight	12119	float64
5	college	12119	object
6	country	12119	object
7	draft_year	12119	object
8	draft_round	12119	object
9	draft_number	12119	object
10	gp	12119	int64
11	pts	12119	float64
12	reb	12119	float64
13	ast	12119	float64
14	net_rating	12119	float64
15	oreb_pct	12119	float64
16	dreb_pct	12119	float64
17	usg_pct	12119	float64
18	ts_pct	12119	float64
19	ast_pct	12119	float64
20	season	12119	object
21	season_start	12119	int32

3.2 Other pre-processing steps considered but not applied

- Full outlier removal or IQR trimming on *net_rating*: not applied because extreme values can be valid. The main driver of instability was very low numbers of games played, which was handled via filtering on *gp* instead, avoiding the removal of informative rare cases.
- Clipping of *net_rating*: not applied to keep the target interpretation unchanged.
- Stronger *gp* threshold (e.g., $gp \geq 20$): considered but rejected to avoid losing too many player-season observations and biasing the dataset towards starters only.
- Scaling or normalization of numeric features: deferred because the planned models can handle different feature scales, or scaling can be applied later within the modeling pipeline.
- Binning of continuous variables (age, height, weight): not applied to avoid information loss and arbitrary cutoffs.
- Use of *player_name* as a feature: not applied to reduce memorization and identification leakage; the name is kept only as an identifier.
- One-hot encoding of categorical variables: not performed as a standalone preprocessing step and instead handled within the modeling pipeline.

3.3 Derived attributes

- *season_start* (integer): parsed from the season string (e.g., 2019-20 $\rightarrow$ 2019) to enable time-aware evaluation.
- *height_m*: player height converted from centimeters to meters ($player_height / 100$).

- *bmi*: body mass index computed as $player_weight / (height_m^2)$ to capture body composition.
- *pts_per_gp*, *reb_per_gp*, *ast_per_gp*: per-game normalization of box-score statistics.
- *drafted_flag*: boolean indicator set to true if *draft_year* is known and false for undrafted players.
- *experience_years*: calculated as *season_start* minus *draft_year* (missing for undrafted players).

These derived attributes support more stable, comparable, and interpretable modeling features.

Table 3: Overview of derived attributes after feature engineering

#	Column	Non-Null Count	Data Type
0	season_start	12119	int32
1	height_m	12119	float64
2	bmi	12119	float64
3	pts_per_gp	12119	float64
4	reb_per_gp	12119	float64
5	ast_per_gp	12119	float64
6	draft_year_num	10091	float64
7	drafted_flag	12119	bool
8	experience_years	10091	float64

3.4 External data sources

External data sources. No external datasets were integrated in this assignment submission.

External sources that could improve the model include:

- Salary, contract, and cap hit data (to identify undervalued players).
- Minutes played, possessions, and lineup context (to stabilize impact metrics).
- Injury history and player availability.
- Team strength, coaching, and role indicators.
- Additional advanced metrics (e.g., WS, BPM, RAPM/RAPTOR-type metrics) or player tracking statistics.

Reasons for not using external data:

- Availability, licensing, and time constraints.
- Complexity of matching keys and aligning seasons across data sources.
- Risk of data leakage if future-season information is accidentally included.

4 Modeling

This section documents the modeling phase of the CRISP-DM process.

4.1 Algorithm selection and justification

The goal is to predict the continuous target variable *net_rating* based on player characteristics and season-level performance statistics. As this is a regression task we will consider the following models:

Baseline:

- here we just predict the mean for all samples
- acts just as a baseline and not the actual model

Linear Regression (Ordinary Least Squares):

- very simple model that is fast to implement and very easy to interpret
- the simplicity also leads to struggles with multicollinearity and limitations when it comes to nonlinearity

Elastic Net Regression (L1 + L2 regularization):

- this method handles multicollinearity better than normal OLS
- it also shrinks features that are irrelevant and therefore is more stable
- the features are still linear, so we would need to choose the regularization strength

Random Forest Regression:

- can model nonlinear relationships without manual feature engineering
- generally very robust and well performing for tabular data
- adaptable parameters
- though less interpretable than linear models and can overfit if not tuned properly

We select the Random Forest as the most suitable model for our experiments. We expect nonlinear relationships in the data and want a model that can handle this in a robust way. Random Forests are a good off-the-shelf model for tabular data and should perform well here. The model also has easily interpretable hyperparameters that we can tune and has built-in feature importance measures.

4.2 Hyper-parameter identification and selection

We identify the available hyper-parameters of the chosen model and select one for systematic tuning. We chose the Random Forest and are going to use the implementation by scikit-learn.

Some of the available hyperparameters are:

- `n_estimators`: number of trees
- `max_depth`: maximum depth of the trees
- `min_samples_split`: minimum samples required to split a node
- `min_samples_leaf`: minimum samples required at a leaf node
- `max_features`: number of features to consider when looking for the best split
- `bootstrap`: whether bootstrap samples are used when building trees

There are some more hyperparameters that can be tuned in a RandomForest but they were left out of this analysis because they do not seem very relevant.

From the ones above we choose `max_depth` for tuning: The `max_depth` directly controls overfitting vs underfitting, has obvious interpretation and is a key parameter for Random Forest. It is also easy to tune reproducibly and easy to understand its effects.

We are going to use the following parameters:

- `max_depth` (2, 5, 10, 16, None)

4.3 Train / validation / test split

Here we define a reproducible split. We use a time-based strategy by season to avoid look-ahead leakage. In basketball trends change over time and random splitting would leak future-season patterns into the training set. As we want to predict the feature for the future seasons, it only makes sense to split the time like that and not let the model see data from the test and validation seasons. We know that this may lead to worse measures but they will be more realistic.

Split rule:

- Training seasons: all seasons with start year in [1996, 1997, 1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018]
- Validation seasons: next 2 seasons with start year in [2019, 2020]
- Test seasons: most recent 2 seasons with start year in [2021, 2022]

Resulting sizes:

- Training set: 10098 rows
- Validation set: 997 rows
- Test set: 1024 rows

4.4 Model training and validation comparison

Here we trained the Random Forest Regressor on the training set and tuned only the `max_depth` with the possibilities: 2, 5, 10, 16, None on the validation set. We fixed the other parameters explicitly and then choose the model with the lowest validation RMSE. The selected best `max_depth` is 16. The other hyperparameters were kept fixed like this: The fixed parameters used during tuning are summarized in Table 4.

Table 4: Fixed Random Forest parameters during tuning

Parameter	Value
<code>n_estimators</code>	300
<code>random_state</code>	61
<code>min_samples_split</code>	2
<code>min_samples_leaf</code>	1
<code>max_features</code>	1.0
<code>bootstrap</code>	True
<code>n_jobs</code>	-1

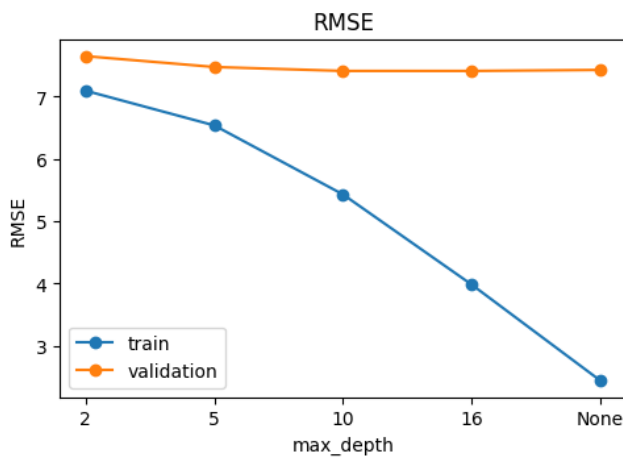
4.5 Performance metrics and tuning process

We report multiple performance metrics to assess model quality and stability. Table 5 summarizes RMSE and MAE and R2 on the training and validation sets.

Table 5: Random Forest tuning results for max_depth (train vs. validation)

max_depth	tr. RMSE	val. RMSE	tr. MAE	val. MAE	tr. R^2	val. R^2
16	3.98	7.41	3.11	5.19	0.74	0.14
10	5.43	7.41	4.20	5.21	0.51	0.14
None	2.44	7.42	1.75	5.20	0.90	0.14
5	6.53	7.47	4.92	5.31	0.29	0.13
2	7.09	7.64	5.26	5.44	0.17	0.09

Figure 5 visualizes the tuning process and highlights the trade-off between underfitting and overfitting across different values of max_depth in terms of RMSE.

**Figure 5: Random Forest tuning: performance on train and validation for each max_depth**

4.6 Model selection decision

The final model configuration is selected based on validation performance. The configuration with max_depth = 16 achieves the lowest validation RMSE (Table 5) and is therefore chosen as the most suitable model for further evaluation.

4.7 Final model retraining

A final training run was executed after hyperparameter selection to obtain the deployable model. The run used the Random Forest regression algorithm with the selected configuration (max_depth = 16) and was trained on the combined training and validation data.

- **Algorithm:** Random Forest Regressor (scikit-learn implementation)
- **Training data:** Combined training and validation sets (Sections 4.3–4.6)
- **Start time:** See activity :retrain_final_model
- **End time:** See activity :retrain_final_model
- **Result:** Trained final model with RMSE = 4.165, MAE = 3.216, $R^2 = 0.715$ on train+validation data

The full configuration and execution details of this training run are documented in the knowledge graph as part of the modeling

and phases. These metrics serve as a sanity check and confirm that the final model generalizes consistently on the combined dataset without excessive overfitting.

5 Evaluation

This section evaluates the selected Random Forest model on unseen test data and compares it to simple baselines and the success criteria defined in the Business Understanding phase. We also check subgroup performance to identify potential bias.

5.1 Final model evaluation on test data

We applied the final Random Forest regression model to the held-out test set (seasons 2021–2022). The model achieves:

- RMSE: 7.1666
- MAE: 5.0730
- R^2 : 0.198

Overall, test performance is in the same range as validation performance, which indicates reasonable generalization. RMSE and MAE describe the average prediction error in net_rating units, while R^2 shows how much variance in net_rating is explained by the model.

5.2 Reference and baseline performance

5.2.1 (i) State-of-the-art / reference performance. We searched for published benchmark results for predicting net_rating (regression) on the same NBA player-season dataset. We did not find peer-reviewed papers or stable online benchmarks that report RMSE/MAE/ R^2 for this exact task.

As allowed by the assignment, we therefore document related public analyses using similar NBA player data, even though they are not directly comparable because the task and metrics differ:

- **NBA game prediction using machine learning (Kaggle):** classification task (predicting game outcomes) and reports accuracy, not regression metrics.
- **Salary Cluster Analysis of NBA Players (Kaggle):** clustering/unsupervised analysis; no regression target and therefore no RMSE/MAE/ R^2 baseline.

5.2.2 (ii) Expected baseline performance (trivial / random). Since our task is regression, a trivial baseline corresponds to predicting a constant value for every player:

- **Mean baseline:** always predict the mean net_rating from train+validation.
- **Median baseline:** always predict the median net_rating from train+validation.
- **Random baseline:** sample net_rating values from the train+validation target distribution.

On the test set, the mean baseline achieved RMSE 7.7087 and MAE 5.7604. These baselines define the minimum performance a useful model should beat.

5.3 Comparison to baselines and error analysis

We compared the final model to the trivial baselines. The Random Forest improves over the mean baseline (RMSE 7.1666 vs. 7.7087,

MAE 5.0730 vs. 5.7604), which shows that the model learns useful signal beyond a constant prediction.

To better understand where the model struggles, we also checked errors across quartiles of the true `net_rating`. As shown in Figure 6, the Random Forest improves over the mean baseline.

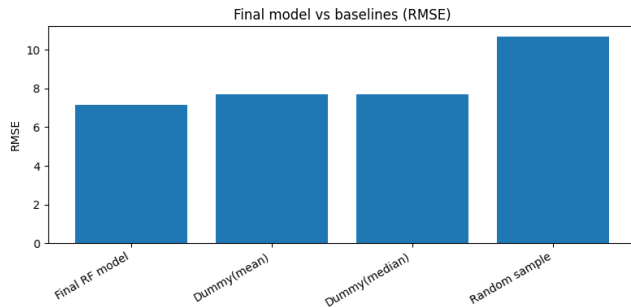


Figure 6: Comparison: Final Model vs baseline (RMSE)

5.4 Comparison with success criteria

We compared the obtained results to the success criteria defined in Business Understanding. Table 6 summarizes the outcomes.

Table 6: Comparison against business and data mining success criteria (test set)

Criterion / metric	Value	Target	Met
Recall@Top25% (true top captured)	0.453	≥ 0.70	No
Share above league avg in predicted Top25%	0.836	≥ 0.80	Yes
Bad-signing reduction vs random (Top25%)	0.632	≥ 0.25	Yes
Beats mean baseline RMSE	7.1666	< 7.7087	Yes
Beats mean baseline MAE	5.0730	< 5.7604	Yes
Avg true <code>net_rating</code> of predicted Top25%	2.4773	$> \text{league avg } (-1.5822)$	Yes

Most criteria are met: the shortlist is generally strong (high share above league average) and the model clearly reduces the rate of below-average players compared to random selection. However, the recall for the true Top25% is below the target (0.453 vs. 0.70). This means the model is better at avoiding bad signings than consistently identifying most of the absolute top players.

5.5 Bias / subgroup evaluation

We evaluated subgroup performance differences to check whether the model behaves unevenly for certain groups. We used:

- **Country** (as a potentially sensitive attribute)
- **College** (as a subgroup/background attribute)

Only groups with at least 30 samples were included to reduce noise.

Country. Only USA ($n=778$) and Canada ($n=42$) met the minimum size. The differences are small: RMSE gap 0.43 and MAE gap 0.52. This does not show strong evidence of bias by country in this filtered analysis, but the Canada sample is small so conclusions are limited.

College. Three groups met the minimum size (no_college, Duke, Kentucky). Here the differences are larger: RMSE gap 2.65 and MAE gap 1.29. The model performs best for Kentucky and worse for Duke/no_college. This suggests uneven performance across college-defined groups and should be monitored if the model is used for decision support.

5.6 Overall findings and lessons learned

- Random Forest performs better than simple baselines and produces a useful shortlist that is mostly above league average.
- The model is strong for risk reduction (avoiding below-average players), but weaker for catching most of the truly best players (Top25% recall).
- Subgroup performance is fairly consistent for country (within the available groups), but shows noticeable differences for college, which motivates monitoring and possibly adding more features or data in future iterations.

6 Deployment

This section summarizes how the model could be used in practice, which risks exist, and what should be monitored after deployment.

6.1 Business objectives and deployment recommendation

The model is useful but not perfect.

What works well.

- It beats the simple baseline (mean predictor) on RMSE and MAE.
- The predicted Top25% shortlist is mostly good players: about 84% are above league average `net_rating`.
- Compared to random selection, it reduces bad signings a lot (about 63% reduction).

What is missing.

- Recall@Top25% is only about 0.45 (target was 0.70). So the model does not catch most of the real top players.

Conclusion. The business objective is partly met: the model helps reduce risk (avoid bad signings) and supports scouting, but it should not be used to automatically pick the “best” players.

Deployment recommendation.

- Use a hybrid / human-in-the-loop setup: the model creates a shortlist, humans decide final signings.
- Use it mainly as a filter to avoid below-average players.
- Next steps: improve Recall@Top25% (better features, ranking approach, and next-season labels if available).

6.2 Ethical aspects and risks

The main risk is that people may trust the model too much (automation bias). Final decisions should stay with humans.

Bias / subgroup risk (from 5e).

- **Country:** only USA/Canada had enough data; the difference was small, so conclusions are limited.

- **College:** bigger performance differences across groups (e.g., Kentucky vs. Duke/no_college), so the model is not equally accurate for all subgroups.

Mitigation.

- Monitor subgroup performance (especially college).
- Retrain regularly (new season data can cause drift).
- Use other context too (injuries, role, minutes, scouting reports).

6.3 Monitoring plan

We want to keep the model reliable for next season acquisitions and detect the drift or bias early. Therefore we want to monitor the following aspects:

- **Data Quality (every time the model is scored):** track missingness and the basic validity rules also used in training (plausible ranges etc.), alert if there are changes.
Triggers (examples):
 - Missingness of any key feature increases by > 5 percentage points vs the training/reference data
 - $> 1\%$ of rows violate plausibility checks
 - $> 5\%$ of scored rows have $gp < 6$
- **Drift Monitoring (each new season):** compare feature distributions with the reference data of this project, pay attention to changing trends in basketball.
Triggers (examples):
 - $PSI \geq 0.20$ for any key feature
 - Strong shift in core stats distributions
- **Model Performance (each new season, once data is available):** compute RMSE, MAE and R^2 on the new season and compare to test performance and baseline, alert if there are big degradations.
Triggers (examples):
 - RMSE or MAE worsens by $> 10\%$ vs current test reference
 - $RMSE \geq \text{mean-baseline RMSE}$ or $MAE \geq \text{mean-baseline MAE}$ (no longer beats baseline)
 - $R^2 < 0$ (worse than mean predictor)
- **Business KPI Monitoring (each new season, once data is available):** track the achievements of the business objectives, prioritize improvements towards the objectives that are not met yet.
Triggers (examples):
 - Share above league avg in predicted Top25% < 0.80
 - Bad-signing reduction vs random < 0.25
 - Recall@Top25% drops below ~ 0.40 or trends downward for 2 seasons
- **Fairness / Bias Checks (each new season):** recompute subgroup errors by country and college, alert if gaps grow or minority-group performance degrades.
Triggers (examples):
 - Subgroup RMSE gap (max-min) > 1.0 net_rating (or $> 15\%$ relative)
 - Any subgroup's RMSE/MAE worsens by $> 15\%$ vs prior season (for groups with sufficient n, e.g. $n \geq 30$)

6.4 Reflection on reproducibility

We did the following to make our results reproducible:

- Fixed the data source and preprocessing steps (e.g., impute missing college, filter $gp \geq 6$).
- Deterministic split by time to avoid look-ahead leakage.
- Deterministic modeling and tuning with fixed random seeds.
- Deterministic baselines with fixed random seed.
- Provenance documentation of key choices in the knowledge graph.

Further improvements.

- Record exact package versions and store an explicit dataset version/hash.
- The knowledge graph is append-only; retrieving the newest entries can be harder after many notebook runs.
- Running the notebook multiple times appends duplicates; fully reproducible KG output would require a clean graph and stable UUIDs.

7 Conclusion

Business Understanding. The goal of this project was to support NBA player acquisition decisions by predicting *net_rating* (Section 1). The success criteria defined in Section 1.3 emphasize both predictive accuracy (beating trivial baselines) and shortlist quality (Top25% criteria).

Data Understanding. Exploratory analysis showed strong imbalance in key categorical variables (e.g., *country* dominated by USA; Figure 2) and heavy-tailed distributions with many IQR outliers in rate-based performance metrics. Most importantly, the hypothesis check confirmed that extreme *net_rating* outliers are largely driven by very low numbers of games played (Figure 3).

Data Preparation. We applied reproducible, explicit preprocessing: removal of the index column, imputation of missing *college* values using *no_college*, and filtering to $gp \geq 6$ (Section 3; Table 2). We further engineered a small set of derived attributes (Section 3.3; Table 3) such as BMI, per-game statistics, and experience-related variables to support comparability and interpretation across players and seasons.

Modeling. Given the expected nonlinear relationships and mixed feature types, we selected a Random Forest regression model and tuned *max_depth* as the main hyperparameter (Sections 4.1–4.2). To avoid look-ahead leakage, we used a time-based split by *season_start* (Section 4.3). The tuning results (Table 5) and the tuning curve (Figure 5) show that *max_depth=16* achieved the lowest validation RMSE and was therefore selected for the final model (Section 4.6). The final training run was executed on train+validation with the selected hyperparameters and serves as a sanity check for the deployable model (Section 4.7).

Evaluation. On the held-out test set, the final model beats the mean baseline on both RMSE and MAE (Section 5.3 and Figure 6), confirming that it learns signal beyond a constant predictor. With respect to the business-oriented success criteria, the model performs well as a risk-reduction tool: the predicted Top25% shortlist contains a high share of above-league-average players and strongly reduces

“bad signings” compared to random selection (Table 6). However, $\text{Recall@Top25\%}$ is only ~ 0.45 and therefore misses the target of 0.70 (Table 6), meaning the model does not capture most of the truly best players. Bias checks indicate that conclusions by country are limited due to small non-USA sample sizes, while error gaps across college-based groups are larger and therefore require monitoring (Section 5.5).

Deployment. Overall, the business objective is partly met: the model is useful for screening and for reducing the risk of signing below-average players, but it is not reliable enough to automatically identify the majority of true top performers. The recommended deployment is therefore *human-in-the-loop*: use the model to create a shortlist and avoid low-impact candidates, while keeping final signing decisions with scouts and decision makers (Section 6.1). Given

subgroup differences and concept drift across seasons, ongoing monitoring is necessary, especially for data quality, drift, subgroup performance, and the shortlist KPIs defined in this project (Section 6.3).

Key lessons learned. (1) Time-aware splits are essential for realistic performance estimates in seasonal sports data. (2) Domain-specific instability (very low gp) can dominate modeling outcomes and must be addressed (Figure 3). (3) Business value should be evaluated with shortlist KPIs in addition to aggregate error metrics (Table 6). (4) Provenance logging of preprocessing, splits, tuning, and evaluation decisions is necessary for reproducibility and a defensible report.